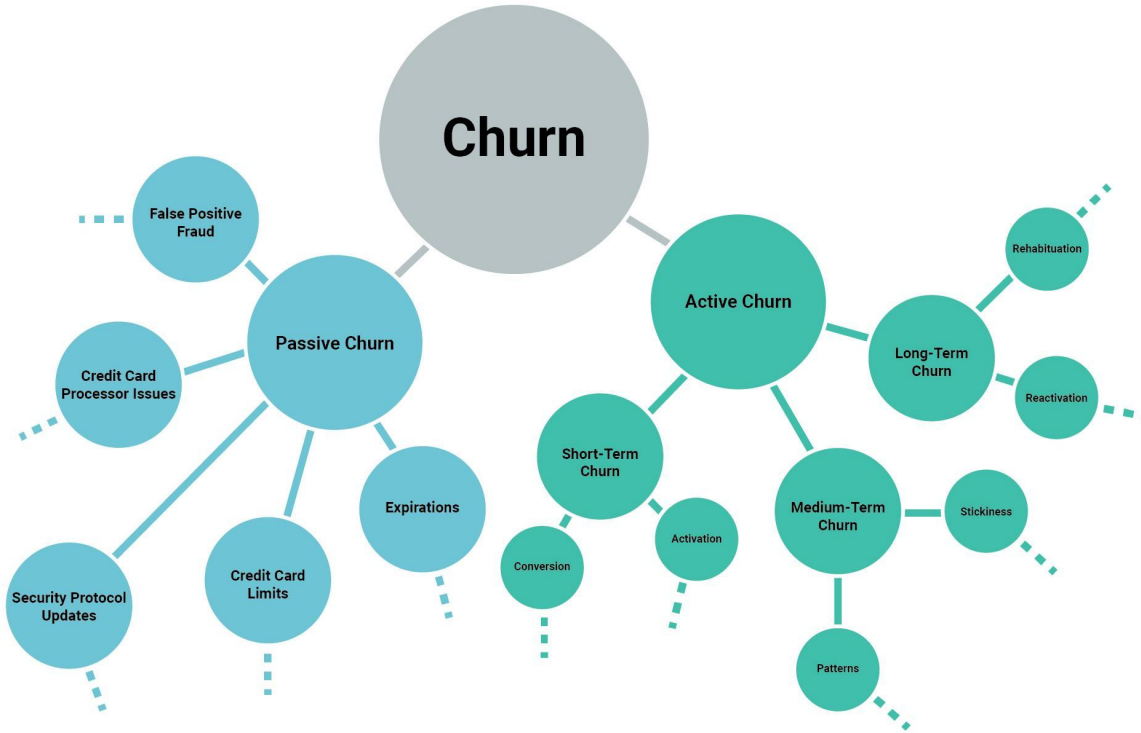


A Rolling Window Framework for Temporal Churn Analysis in SaaS

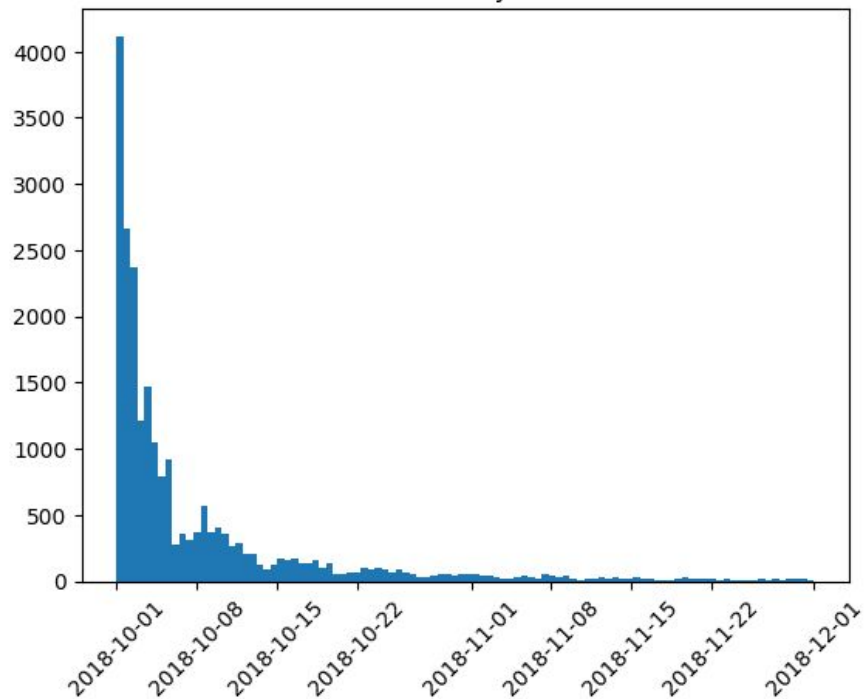
Henry Ding

Motivation

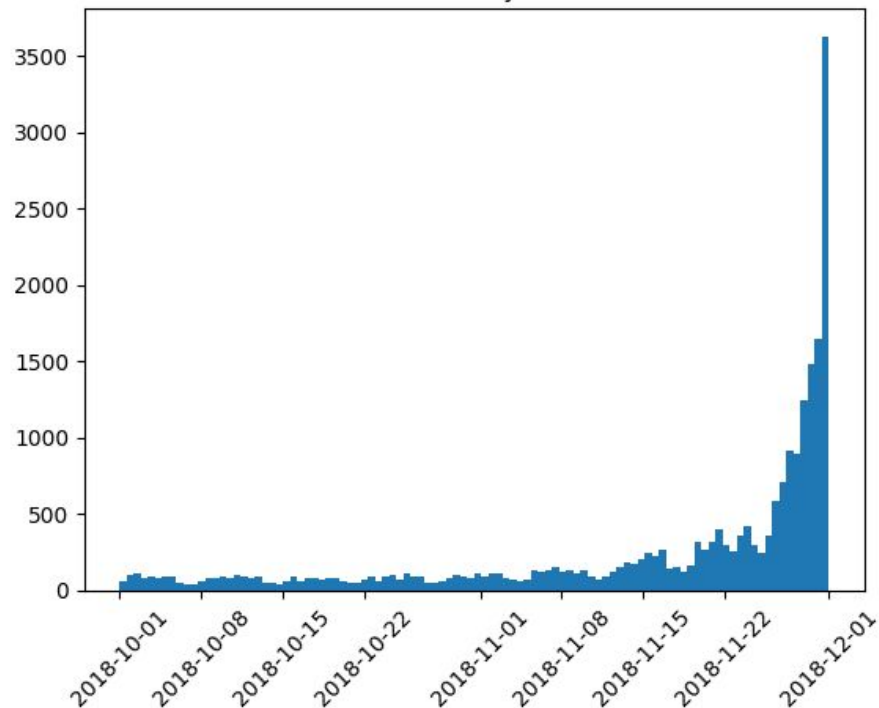


Motivation

First Activity Date



Last Activity Date



Literature Review

- Churn is hard to predict
 - Studies utilize ensemble-fusion (compilation of machine learning models)
 - BiLSTM-CNN model
 - 95.35% accuracy and a 96.96% F1-score, outperforming other models
 - Not enough data
- Need to measure profitability of retention rather than percent predicted and retained
 - A study improved profitability metrics, achieving a Gini coefficient of 0.142 and a Top Decile Lift of 1.291
 - Bagging and Boosting study tackled this issue through balanced sampling schemes and bias correction techniques
-

Literature Review

- Need to spot customer churn quickly to act on
 - Devise a temporal method to quickly predict next week churn
- Limited studies on passive churn
 - Study quiet churn

Methodology: Define Churn

$$\text{QuietChurn}_i(t) = \begin{cases} 1 & \text{if } A_i(t) = 1 \text{ and } \sum_{k=1}^{\Delta t} A_i(t+k) = 0 \\ 0 & \text{otherwise} \end{cases}$$

- t = current time period (e.g., today or this week)
- $A_i(t)$ = activity indicator (1 if user i is active at time t , 0 otherwise)
- Δt = future window (e.g., 7 days)

Methodology: Rolling Counts

$$\text{Count}_t = \sum_{i=t-n+1}^t \mathbb{1}_{x_i \neq \text{NaN}}$$

- Count_t : Non-NaN count at time t
- $\sum_{i=t-n+1}^t$: Sum over window size n
- $\mathbb{1}_{x_i \neq \text{NaN}}$: Indicator for non-NaN value
- x_i : Value at index i
- n : Window size
- t : Current time/index

Methodology: Rolling Std

$$\sigma_t = \sqrt{\frac{1}{n} \sum_{i=t-n+1}^t (x_i - \mu)^2}$$

- μ = rolling mean over the same window
- x_i = value at time i

Methodology: Algorithmic Structure

For each $i \in \{0, \dots, N-2\}$:

$$X_{\text{train}} = \{x^{(j)} \mid \text{user } j \text{ in week } w_i\}$$

$$y_{\text{train}} = \{y^{(j)} \mid \text{user } j \text{ in week } w_i\}$$

$$X_{\text{test}} = \{x^{(j)} \mid \text{user } j \text{ in week } w_{i+1}\}$$

$$y_{\text{test}} = \{y^{(j)} \mid \text{user } j \text{ in week } w_{i+1}\}$$

$$f_i = \text{Train model on } (X_{\text{train}}, y_{\text{train}})$$

$$\hat{y}_{\text{test}} = f_i(X_{\text{test}})$$

- w_i : the **training week**
- w_{i+1} : the **test week**
- $x^{(j)}$: feature vector for user j
- $y^{(j)}$: churn label for user j
- f : your ML model
- $\hat{y}^{(j)}$: predicted label

Methodology: Final Equation

$$\hat{y}^{(j)} = \sigma \left(\beta_0 \beta_1 \left(\frac{1}{7} \sum_{k=0}^6 a^{(j)}(t-k) \right) \beta_2 \left(\sum_{k=0}^{20} a^{(j)}(t-k) \right) \beta_3 \left(\sum_{k=0}^{27} \mathbb{1}(a^{(j)}(t-k) > 0) \right) \beta_4 x_4^{(j)} \beta_5 x_5^{(j)} \right)$$

- $\hat{y}^{(j)}$ = predicted churn probability for user j
- $\sigma(\cdot)$ = sigmoid function
- β_0 = intercept
- $\beta_1 \dots \beta_5$ = learned weights from model training

Methodology: Profitability for Customer over Time Window

$$\text{Value Lost}_t^{(i)} = \sum_{j=t-n+1}^t \text{Value}_j^{(i)}$$

- $\text{Value Lost}_t^{(i)}$: Total value lost for customer i at time t .
- $\text{Value}_j^{(i)}$: Value generated by customer i at time j (in this context, the value you lose if the customer churns).
- n : Time window size (e.g., days, months, etc.).
- t : Current time step.
- j : Index representing time steps within the window.

Methodology: Total Value for Customer

$$\text{Total Value}^{(i)} = \sum_{j=1}^T \text{Value}_j^{(i)}$$

- $\text{Total Value}^{(i)}$: The total value generated by customer i over the entire period.
- T : Total number of time steps (e.g., total months, days, or value instances).

Methodology: Weighted Loss Function for High-Value Customers

$$\mathcal{L} = \sum_{i=1}^N w^{(i)} \cdot \mathcal{L}_{\text{base}}^{(i)}$$

- $\mathcal{L}$: Total loss function for the model.
- N : Total number of customers in the dataset.
- $w^{(i)}$: Weight assigned to customer i , which could be based on their **Total Value** or **Value Lost**.
- $\mathcal{L}_{\text{base}}^{(i)}$: Base loss function for customer i , typically the binary cross-entropy or mean squared error depending on the task.

Methodology: Model Output

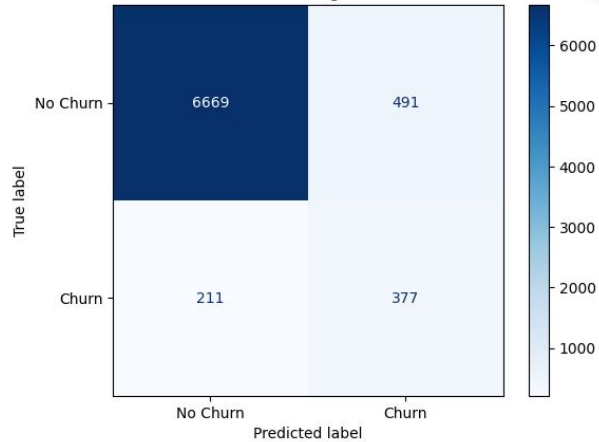
$$\hat{y}_t^{(i)} = \text{Churn Probability}_t^{(i)}$$

- $\hat{y}_t^{(i)}$: Predicted churn probability for customer i at time t .

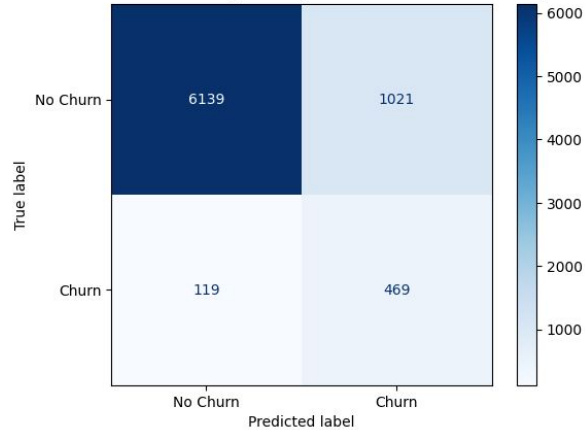
Empirical Results

	churn_label	precision	recall	f1-score	support
Week 2024-11-04 00:00:00 Model: Random Forest	1	0.48	0.37	0.41	588
Week 2024-11-04 00:00:00 Model: Logistic Regression	1	0.14	0.92	0.25	588
Week 2024-11-04 00:00:00 Model: Gradient Boosting	1	0.43	0.64	0.52	588
Week 2024-11-04 00:00:00 Model: Random Forest	1	0.45	0.36	0.40	588
Week 2024-11-04 00:00:00 Model: XGBoost	1	0.37	0.71	0.49	588
Week 2024-11-04 00:00:00 Model: LightGBM	1	0.31	0.80	0.45	588
Week 2024-11-04 00:00:00 Model: SVM	1	0.13	0.96	0.22	588

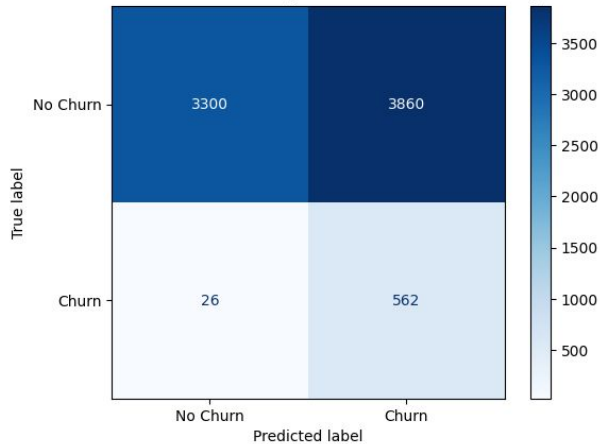
Confusion Matrix - Gradient Boosting (Week 2024-11-04 00:00:00)



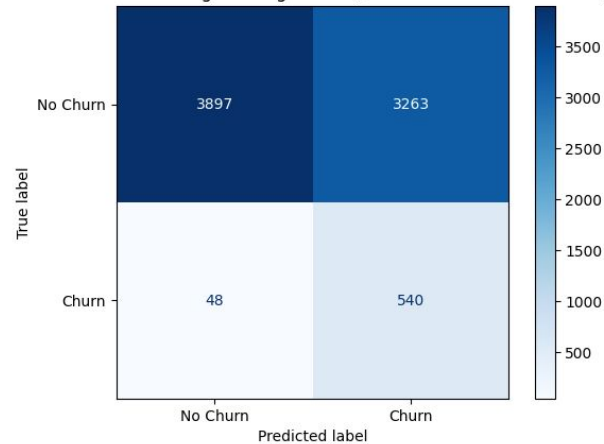
Confusion Matrix - LightGBM (Week 2024-11-04 00:00:00)



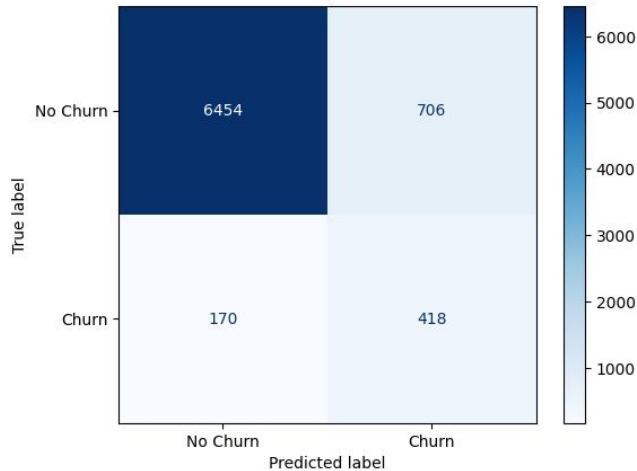
Confusion Matrix - SVM (Week 2024-11-04 00:00:00)



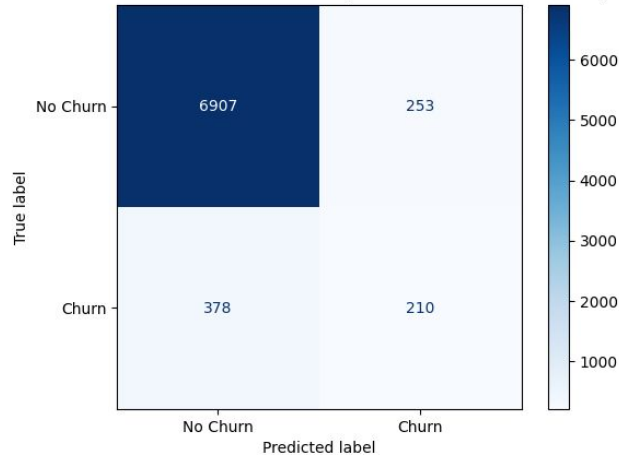
Confusion Matrix - Logistic Regression (Week 2024-11-04 00:00:00)



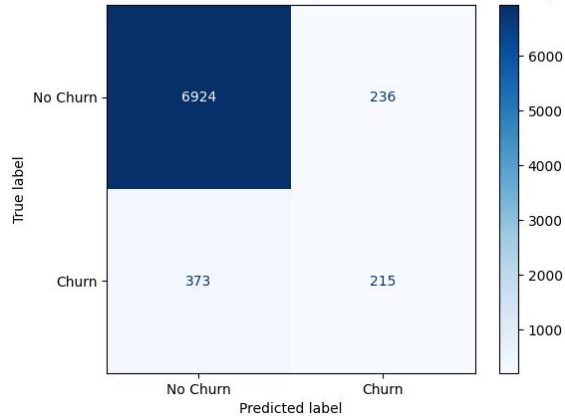
Confusion Matrix - XGBoost (Week 2024-11-04 00:00:00)



Confusion Matrix - Random Forest (Week 2024-11-04 00:00:00)



Confusion Matrix - Random Forest (Week 2024-11-04 00:00:00)



Conclusion

- Overall it seems to be harder to predict quiet or passive churn than previous models. The f1-score is currently at 55% at the highest.
- Next steps: Would be to factor in the weighted loss function for high value customers and attach a probability. This would sort all the customers in highest likelihood of churning.